

Program 1

Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist.

```
CREATE OR REPLACE TRIGGER Prevent_Parent_delete
BEFORE DELETE ON Parent
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM child
    WHERE parent_id = :OLD.parent_id;

    IF v_count > 0 THEN
        RAISE -APPLICATION-ERROR(-20001, 'Cannot delete Parent
        record: Child records exist,');
    END IF;
END;
/
```



Program 2

Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

```
CREATE OR REPLACE TRIGGER check duplicate  
BEFORE INSERT OR UPDATE ON employees  
FOR EACH ROW
```

```
DECLARE
```

```
    v_count NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_count  
    FROM employees  
    WHERE email = : New email;
```

```
    IF v_count > 0 THEN
```

```
        RAISE APPLICATION_ERROR(-20002, 'Duplicate email not allowed.');
```

```
    END IF;
```

```
END;
```


Program 3

Write a code in PL/SQL to create a trigger that restricts the insertion of new rows if the total of a column's values exceeds a certain threshold.

```
CREATE OR REPLACE TRIGGER limit_total_Sales
BEFORE INSERT ON Sales
FOR EACH Row
Declare
    v-total NUMBER;
    v-limit CONSTANT NUMBER := 100000;
BEGIN
    SELECT SUM (amount) INTO v-total FROM Sales;
    IF v-total + :New .amount > v-limit THEN
        RAISE -APPLICATION -ERROR( 20003, ' ( amount :-
        total Sales exceed limit, ' )
    END IF;
END;
```



Program 4

Write a code in PL/SQL to design a trigger that captures changes made to specific columns and logs them in an audit table.

```
CREATE TABLE emp_audit (  
    emp_id NUMBER,  
    old_salary NUMBER,  
    new_salary NUMBER,  
    changed_on DATE  
);
```

```
CREATE OR REPLACE TRIGGER log_salary_change  
AFTER UPDATE OF SALARY ON employees  
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO emp_audit (emp_id, old_salary, new_salary,  
                           changed_on )  
VALUES (:OLD.employee_id, :OLD.salary, :New_salary, SYSDATE);
```

```
END;
```

```
/
```


Program 5

Write a code in PL/SQL to implement a trigger that records user activity (inserts, updates, deletes) in an audit log for a given set of tables.

```
CREATE TABLE audit_log (  
    username VARCHAR(30),  
    action_type VARCHAR(10),  
    table_name VARCHAR2(30),  
    action_date DATE  
);
```

```
CREATE OR REPLACE TRIGGER record_user_activity  
AFTER INSERT OR UPDATE OR DELETE ON employees  
BEGIN
```

```
INSERT INTO audit_log ( username, action_type, table_name, action_date )  
VALUES (USER,  
        CASE  
            WHEN INSERTING THEN 'INSERT'  
            WHEN UPDATING THEN 'UPDATE'  
            WHEN DELETING THEN 'DELETE'  
        END,  
        'EMPLOYEES',  
        SYSDATE);  
END;
```


Program 7

Write a code in PL/SQL to implement a trigger that automatically calculates and updates a running total column for a table whenever new rows are inserted.

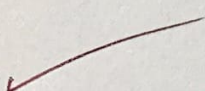
```
CREATE OR REPLACE TRIGGER Calc-running-total
BEFORE INSERT ON Sales
FOR EACH ROW
DECLARE
    v-total Number;
Begin
    select NVL (SUM (amount), 0) into v-total from Sales;
    :NEW - running-total := v-total + :new.amount;
END;
```


Program 8

Write a code in PL/SQL to create a trigger that validates the availability of items before allowing an order to be placed, considering stock levels and pending orders.

```
CREATE OR REPLACE TRIGGER check-item-stock
BEFORE INSERT ON orders
FOR EACH ROW
DECLARE
    v-stock NUMBER;
BEGIN
    SELECT stock-qty INTO v-stock
    FROM items
    WHERE item-id = :new.item-id;

    IF v-stock < :new.order-qty THEN
        RAISE -APPLICATION_ERROR (-20008, 'Not enough
        stock to place the order,');
    END IF;
END;
/
```



Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	<i>[Signature]</i>